H03 – NestJS CRUD

A Users REST Service Built with NestJS

Bara S. Rohmani
NRP: 5025241144
Framework-Based Programming, Informatics Engineering, ITS
2 November 2026

1 Introduction

This assignment asks for the same kind of users REST service as H01, but built with NestJS instead of the raw `node:http` module. The service supports the four CRUD operations on a `users` resource, validates incoming data, and answers in JSON. The second goal is to compare this implementation with my pure Node.js version and decide which one is better, and why.

The implementation is spread across 16 TypeScript files under `src/`, totaling 244 lines when generated specs are included. Excluding those specs, the application code is 184 lines across 13 files. It passes all 43 tests in `test/app.e2e-spec.ts` when running `pnpm test:e2e`.

2 Environment and Tools

Item	Detail
Runtime	Node.js v22.19.0 on Windows (Git Bash)
Package manager	pnpm 12.16.1
Framework	NestJS 12.3.2 (`@nestjs/common`), scaffolded with `@nestjs/cli` 12.0.11 (CommonJS, Jest)
Validation	`class-validator` 0.15.1 and `class-transformer` 0.5.1
Testing	Jest 30.5.2 with `@nestjs/testing` 12.3.2 and `supertest` 7.3.1
Storage	In-memory array inside `DatabaseService`

In H01 I used SQLite so that data survives a restart. Here the e2e tests call `resetData()` on a `DatabaseService`, and the assignment does not require persistence, so I store users in an array. This is a real difference between the two projects and it affects the comparison, which I discuss in the comparison section.

3 Core Concepts Used

Before the steps, here are the NestJS concepts that replace the manual work from H01.

- **Module** (`@Module`): groups related controllers and providers. A provider is only visible to another module if its owner exports it and the other module imports it.
- **Controller** (`@Controller`): maps URLs and HTTP methods to methods with decorators such as `@Get` and `@Post`.
- **Provider / Service** (`@Injectable`): holds logic and data. Nest creates it and passes it to whoever asks for it in a constructor. This is *dependency injection*.
- **DTO and `ValidationPipe`**: a DTO is a class describing the body of a request. Decorators from `class-validator` state the rules, and `ValidationPipe` enforces them before the handler runs.

- **Pipe** such as `ParseIntPipe`: converts or validates a single value, for example turning the path parameter `"1"` into the number `1` and rejecting `"abc"` with a 400 response.
- **Exception**: throwing `NotFoundException` is enough for Nest to produce a 404 response.

4 Implementation Steps

4.1 Step 1: Scaffolding the project

I installed the CLI with `pnpm add -g @nestjs/cli` and created the project with `nest new`. The details of what went wrong here are in the section on problems below. After the scaffold I installed `class-validator`, `class-transformer`, `@nestjs/testing`, and `supertest`, and copied the provided `app.e2e-spec.ts` and `jest-e2e.json` into `test/`.

4.2 Step 2: Generating the resource

`nest generate resource users` with the REST API and CRUD entry points options created the `users` folder with a module, controller, service, two DTOs, and an entity. The generator registers `UsersModule` in `AppModule` automatically, which is one of the small conveniences compared with H01 where I wired everything by hand.

4.3 Step 3: A database provider

The test file imports `DatabaseService` from `src/database/database.service.ts` and calls `resetData()` on it before each test, so the storage had to live in its own provider. I created `DatabaseService` (an array and a counter for the next ID) and a `DatabaseModule` that exports it. `UsersModule` imports `DatabaseModule`, so `UsersService` can receive the database through its constructor.

```
@Injectable()
export class DatabaseService {
  users: User[] = [];
  nextId = 1;

  resetData(): void {
    this.users = [];
    this.nextId = 1;
  }
}
```

4.4 Step 4: Entity and DTOs

The `User` entity has the same fields as in H01: `id`, `name`, `age`, `isAdmin`, `createdAt`, and `updatedAt`. `CreateUserDto` replaces `parseUserInput` from H01. Each rule is a decorator, and each decorator carries a custom `message`, because the tests compare against exact message texts.

```
export class CreateUserDto {
  @Transform(({ value }) => (typeof value === 'string' ? value.trim() : value))
  @IsString({ message: 'Name must be a string' })
  @IsNotEmpty({ message: 'Name is required and must be a non-empty string' })
  @MaxLength(100, { message: 'Name must be 100 characters or less' })
  name: string;

  @IsInt({ message: 'Age must be a number' })
  @Min(0, { message: 'Age must be between 0 and 150' })
```

```
  @Max(150, { message: 'Age must be between 0 and 150' })
  age: number;

  @IsOptional()
  @IsBoolean({ message: 'isAdmin must be a boolean value' })
  isAdmin?: boolean;
}
```

`UpdateUserDto` extends `CreateUserDto` instead of using the generator's `PartialType`. The tests require `PUT` to behave as a full replacement: an empty name must be rejected and a missing `isAdmin` must reset to `false`. With `PartialType`, every field would be optional and those tests would fail.

4.5 Step 5: Service

`UsersService` receives `DatabaseService` through its constructor and implements `create`, `findAll`, `findOne`, `update`, and `remove`. On creation, `createdAt` and `updatedAt` get the same value. On update, only `updatedAt` changes. A missing user throws `NotFoundException({ error: 'User not found' })`. When an exception receives an object, Nest uses that object directly as the response body, which gives exactly `{ error: 'User not found' }` as the tests expect. A plain string would have been wrapped into Nest's default format and the test would fail.

4.6 Step 6: Controller

I removed the generated `@Patch` route and wrote `@Put(':id')`, because the tests use `PUT` and expect `PATCH /users/1` to return 404. All routes that take an ID use `@Param('id', ParseIntPipe)`. I did not write any status codes: Nest answers 201 for `@Post` and 200 for the other routes by default, which matches the tests.

4.7 Step 7: `initApp` and `main.ts`

The e2e test builds its own application with `Test.createTestingModule`, so `main.ts` is never executed during the tests. For that reason the test file imports a function called `initApp` from `src/app.init.ts`. I put the global setup there, and `main.ts` calls the same function, so the real server and the tests run with identical settings.

```
export function initApp(app: INestApplication): void {
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
  app.enableCors();
}
```

`whitelist: true` removes properties that are not in the DTO, and `transform: true` is needed for `@Transform` (trimming the name) to take effect. I deliberately did not enable implicit type conversion, because it would turn the string `"true"` into a boolean and the test that expects a 400 for `isAdmin: 'true'` would pass validation by mistake.

5 Testing

`pnpm test:e2e --runInBand` finishes with **43 out of 43 tests passing**. The suite covers all four CRUD operations, validation boundaries, ID handling, CORS, 404 handling, and integration scenarios such as a full create-read-update-delete cycle. The number is lower than the 46 tests in H01 because these are two different test suites written for two different assignments, so the numbers cannot be compared directly.

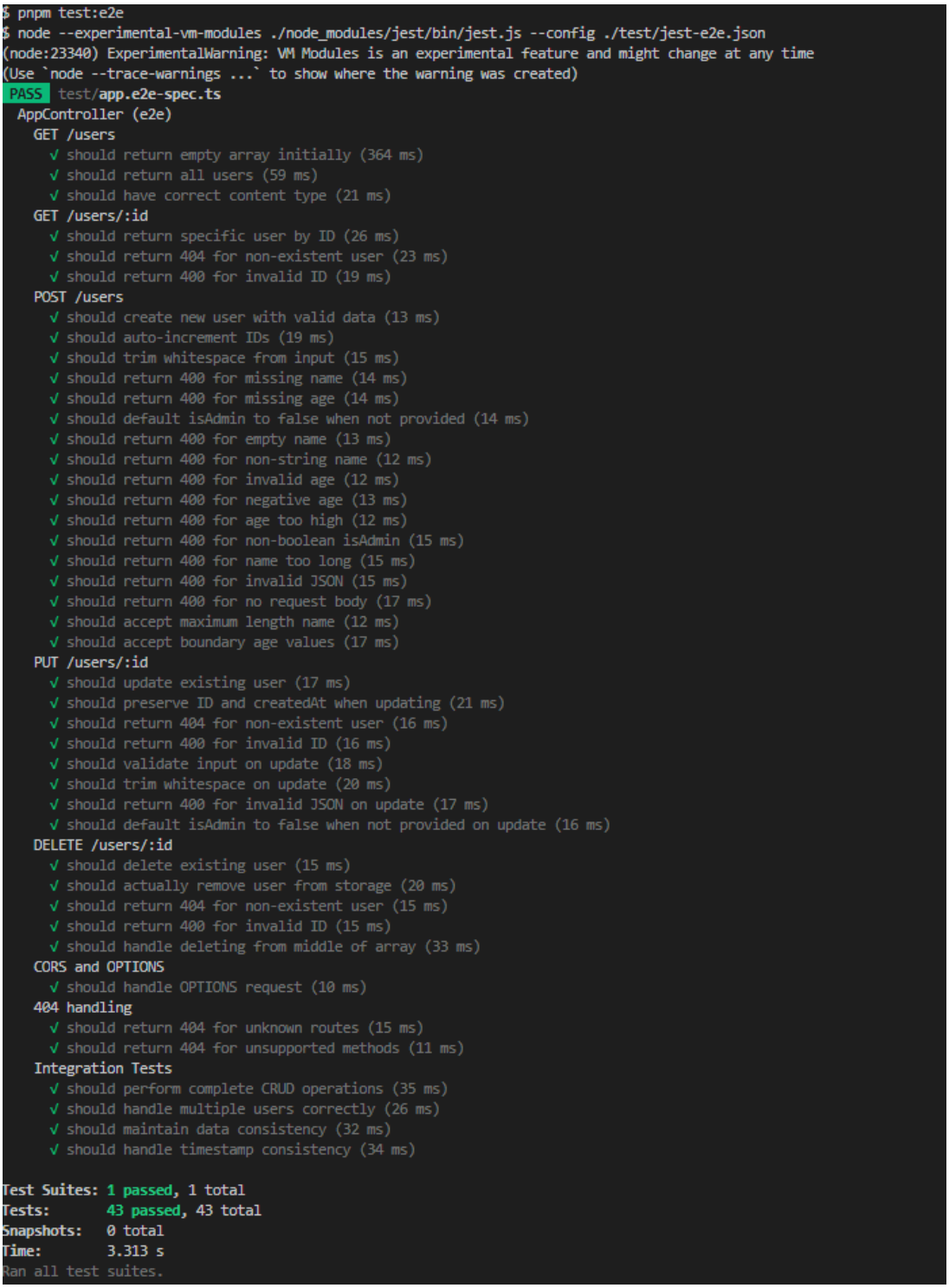


Figure 1: Result of `pnpm test:e2e`.

6 Problems Found

6.1 Scaffolding failed with “A merge conflicted”

Running `nest new .` in the assignment folder failed repeatedly with `A merge conflicted on path "/package.json"`, then `"/test/app.e2e-spec.ts"`, then `"/test/jest-e2e.json"`. The message changed depending on the module system I selected, which made it look like a problem with the CLI version or with ESM versus CommonJS. I first tried reinstalling the CLI, which changed nothing.

The real cause was that the template folder already contained files that the Nest schematic also wants to create. The schematic refuses to overwrite existing files and stops at the first conflict it finds. Its reported paths were `/package.json`, `/test/app.e2e-spec.ts`, and `/test/jest-e2e.json`. I kept the assignment files, cleared the conflicting paths, ran `nest new` again with CommonJS and Jest (because the assignment uses `jest-e2e.json` and `ts-jest`), and restored the two test files in `test/`.

The lesson is that a scaffolding tool assumes an empty target folder. In H01 there was no such tool, so this kind of problem did not exist.

6.2 Generated unit-test scaffolds need updating after adding `DatabaseService`

`DatabaseService` is not part of the NestJS-generated resource. I added `src/database/` with `DatabaseService` and `DatabaseModule` because the provided e2e test imports the service and needs a resettable in-memory store. That addition also made `UsersService` depend on `DatabaseService`, but the generated `users.controller.spec.ts` and `users.service.spec.ts` still register `UsersService` without its new dependency. As a result, `pnpm test --runInBand` fails in those two generated unit-test scaffolds. The e2e suite passes because it imports the complete `AppModule`. The unit specs would need to import `DatabaseModule` or provide a `DatabaseService` test double; I left them unchanged because the assignment requires the e2e suite, not the unit suite.

7 Manual Testing

I started the server with `pnpm start:dev` and checked representative requests. These results are transcribed from the HTTP responses rather than screenshots.

Request	Status	Observed result
`GET /users`	200	An empty JSON array before creating a user.
`POST /users` with a valid user	201	The created user is returned with an ID and matching `createdAt`/`updatedAt` timestamps.
`GET /users/abc`	400	Nest returns a `Bad Request` body because the ID is not numeric.
`GET /users/`	200	The trailing slash is treated as the collection route and returns an empty JSON array.
`POST /users` with malformed JSON	400	The JSON parser error is returned in Nest's standard `message`, `error`, and `statusCode` fields.

7.1 Behavior the framework decides for me

Two behaviors are not covered by the tests, so I checked them manually and compared them with H01.

- `GET /users/` with a trailing slash: in H01 this returned 400 because my regular expression treated it as an item with an empty ID. In NestJS it returned 200 with `[]`; Express treats the trailing slash as the collection route.
- `POST /users` with malformed JSON and `Content-Type: application/json`: in H01 the response was 400 "Invalid JSON". In NestJS it returned 400 with a parser message such as `"Unexpected end of JSON input"` and the fields `error: "Bad Request"` and `statusCode:`

400. The test named “invalid JSON” sends a plain string through supertest, so it does not exercise this same malformed-JSON parser path.

8 Comparison with Pure Node.js (H01)

8.1 Mapping of responsibilities

Work in H01 (raw node:http)	Same work in NestJS (H03)
Regex USERS_PATH and createRouter	@Controller('users') with @Get, @Post, @Put, @Delete
switch on req.method	One method per HTTP decorator
readBody and readJsonBody	@Body() and the built-in body parser
parseId	@Param('id', ParseIntPipe)
parseUserInput	DTO with class-validator decorators and ValidationPipe
HttpError and toErrorResult	Built-in exceptions such as NotFoundException
RESPONSE_HEADERS and sendJson	Automatic JSON responses and app.enableCors()
createMyServer and index.ts	NestFactory, AppModule, dependency injection

8.2 Measured differences

Aspect	H01 (raw Node.js)	H03 (NestJS)
Source files	1 (server.ts) plus index.ts	13 application TypeScript files (16 including generated specs)
Lines of code	302 in server.ts	184 application lines (244 including generated specs)
Storage	SQLite (better-sqlite3)	In-memory array
Declared direct dependencies	1 runtime + 7 development (8 total)	8 runtime + 19 development (27 total)

8.3 Fairness of the comparison

The two projects are not identical, and this weakens any numeric comparison. H01 uses SQLite and H03 uses an array, so H01 contains repository and SQL code that H03 does not need. The error formats also differ: H01 returns a details array, while NestJS returns a message array. The test suites are different too. Because of this, I use the line counts only as a rough indication and I base my verdict mainly on where the work happens and who controls it.

8.4 Where NestJS is better

- **Less repetitive plumbing.** Routing, body parsing, status codes, JSON serialization, and CORS were about half of the code I wrote in H01. In NestJS each of these became a decorator or one line of configuration.

- **Declarative validation.** The rules in `CreateUserDto` can be read like a specification. In H01 they were an imperative function that collected errors manually, and any new field meant more branching code.
- **Clear structure.** Controller, service, and database each have a fixed place. In H01 the layers were also separated, but only because I chose that design, nothing enforced it.
- **Testability through dependency injection.** `resetData()` could be called from the test because the database is a provider that the test can fetch from the module. In H01 I had to expose a separate reset function from the server factory.

8.5 Where NestJS is worse

- **Hidden behavior.** Many things happen without any code that I wrote: trailing slashes, JSON parsing errors, and the default error body are decided by Express and Nest. The trailing slash case above is a concrete example where the framework's default differs from the behavior I deliberately designed in H01.
- **More files and concepts for a small task.** Module, controller, service, DTO, entity, and a separate database module must be understood before anything works. For a service of this size, the raw version is easier to read from top to bottom.
- **Learning curve and error messages.** Decorators and dependency injection are hard to trace. The scaffolding conflict showed how a tooling problem can block progress before any actual code is written.
- **Dependency weight and build tooling.** H03 declares 27 direct dependencies (8 runtime and 19 development), compared with H01's 8 (1 runtime and 7 development). These counts do not measure the full transitive `node_modules` tree, but they show the additional framework and build-tooling surface. H01 uses the built-in `node:http` for its web layer.

8.6 Verdict

For this assignment alone, the raw Node.js version is perfectly adequate and has less to learn. Even so, I judge NestJS to be the better choice for any service that will grow beyond a few resources. The reason is not that it has fewer lines, since part of that difference comes from the storage change. The reason is that it moves the repetitive, error-prone work (routing, parsing, validation, error mapping) into tested, declarative building blocks, so that my own code is mostly business rules. The price is less visibility and control: I now depend on defaults that I have to learn and verify, as the trailing slash case shows. H01 is what made this price understandable, because I know what each decorator replaces.

9 Limitations

- **Data is not persistent.** The array lives in memory, so all users disappear when the server stops. H01 used SQLite and did not have this problem.
- **No authentication or authorization.** Anyone can create, change, or delete any user, and a client can set `isAdmin: true` for itself.
- **No pagination.** `GET /users` returns every user.
- **`PUT` replaces all fields.** A request that omits `isAdmin` silently resets it to `false`. `PATCH` is not supported and returns 404, as the tests require.
- **`findAll` returns the internal array.** Any code that receives it could modify the stored data. Returning a copy would be safer.
- **Generated unit tests were left in `src/` but are not maintained.** The generated `users` specs do not provide the `DatabaseService` dependency, so `pnpm test` currently fails in those two suites. The assignment's `pnpm test:e2e` suite passes.

- **Request body size is limited only by Nest's defaults.** I did not configure it explicitly.

10 Conclusion

Building the same service twice shows what a framework changes. In H01 every step of a request was visible and written by me. In H03 most of those steps became decorators and configuration, and my code shrank to the data model, the validation rules, and the service logic. The result is easier to extend, and it passes all 43 e2e tests. It is also less transparent, so I had to check the framework's defaults manually. My conclusion is that NestJS is worth its overhead for services that will grow, while raw `node:http` remains a good way to learn what is happening underneath.